

Ahmed Corrugation Machines

Process Map, Agent Architecture and Scope of Work

Prepared for: the AI engineer taking ownership of this build

Prepared by: Muhammad Qasim

Date: 27 August 2026

Accompanies: the legacy codebase ([Factory Management System First Deployment](#))

Contents

#	SECTION
0	How to read this document
1	The business — vocabulary, catalogue, pricing, customers, money model
2	The processes, as they run today — twelve processes
3	The core rule: one transaction, many effects
4	Target system architecture
5	The agents — nine agents, defined
6	Engineering rules for the agent layer
7	Adoption: running in parallel
8	Delivery plan
9	What to collect from the factory before writing code
10	Definition of done
A	Verified pricing formulas
B	Legacy codebase map and known defects
C	Open questions

0. How to read this document

This is a scope-of-work document, not a specification to implement line by line. It gives you three things:

1. **The business knowledge** — what this factory does, in its own vocabulary, and how money and material actually move through it. This is the part that is hard to acquire and easy to get wrong.
2. **The process map** — every process the business runs today, how it is done manually, and what automation each one needs.
3. **The agent architecture** — the agents you are expected to build, what each one is allowed to do, and where a human must stay in the loop.

You own this from design to delivery. The architecture below is a considered starting point derived from a direct reading of the legacy code and the factory's own data, but you are expected to challenge it where you find something better. What is **not** open for negotiation is in Section 6: the safety and human-in-the-loop rules.

Notation used throughout

MARKER	MEANING
[Verified]	Confirmed by reading the legacy source code or the factory's spreadsheets. Trust it.
[Verify]	Inferred. Plausible, but you must confirm with the factory before building on it.
[Decision]	A design decision with a recommendation and a rationale. Yours to make, but read the rationale first.

The single most important principle

The factory keeps running manually the entire time you are building.

Nothing you deliver may block a sale. If an agent cannot parse something, if stock looks wrong, if the internet is down — the staff must still be able to raise a bill and let goods out of the gate. Every automated path needs a manual override sitting next to it. A system that is occasionally wrong but never blocks will be adopted; a system that is always right but sometimes stops the factory will be abandoned, which is exactly what happened to the 2022 attempt.

1. The business

1.1 What the business does

Ahmed Corrugation Machines trades and converts **corrugated packaging paper**. The business has three revenue activities:

1. **Trading paper in bulk** — buying and reselling paper in *reels* (large rolls sold by weight) and *rolls* (sold by count, priced by width).
2. **Converting paper into product** — cutting and processing reels or *totay* into *packets* of sheets at a customer's specified size and grammage. This is where the corrugation machines earn their keep.
3. **Trading by-products and waste** — *raddi*, *jutta* and *nali*, all sold by weight.

Customers are typically box makers and small packaging units. Some pay cash at the counter; others run a monthly account.

1.2 Vocabulary — learn this first

You cannot design the screens or the agent prompts without this vocabulary, because it is what the staff will type and say. Do not translate it away.

TERM	MEANING
Reel	A large roll of paper sold by weight in kilograms . Identified by paper type, size (width) and weight. [Verified]
Roll	Paper sold by count , priced using its size (width). Stock is a simple count per type and size. [Verified]
Tota / Totay	Part-used or offcut reel stock, sold by weight . Tracked separately from full reels. [Verified]
Packet	A bundle of cut sheets, made to order from a reel or a tota. Priced by sheet area and grammage. [Verified]
Bareek	“Thin” — a finer variant of a paper type, with its own rate. Every one of the nine base paper types has a Bareek counterpart. [Verified]
Raddi	Waste paper / scrap, sold by weight. [Verified]
Jutta	A by-product sold by weight. Exact nature to be confirmed. [Verify]
Nali	Paper tube or core, sold by weight. [Verify]
Grammage	Paper weight in grams per square metre. Drives packet pricing. [Verified]
Size	Width of the paper, in inches. Observed range 17–22. [Verified]
Rent	A charge with its own column in every customer ledger. Its meaning is not documented anywhere in the code. [Verify — see Section 9]
Gate Pass	The document authorising goods to physically leave the premises. May be issued before the bill. [Verify — current practice]

1.3 The materials catalogue

Nine base paper types, each with a Bareek variant — **eighteen in total**. **[Verified]**

BASE TYPE	BAREEK VARIANT	TYPICAL USE
Fluting	Fluting Bareek	Middle liner of corrugated board, ~85–105 gsm
Super Fluting	Super Fluting Bareek	Higher grade fluting
L1	L1 Bareek	Recycled base liner, ~140–150 gsm, waterproof
L2	L2 Bareek	Second-grade liner
Test Liner	Test Liner Bareek	Outer liner board
Box Board 2.5 No	Box Board 2.5 No Bareek	Board grade
Box Board 3 No	Box Board 3 No Bareek	Board grade
Local Kraft	Local Kraft Bareek	Domestically produced kraft
Imported Kraft	Imported Kraft Bareek	Imported kraft, premium

Each type carries a rate, a description and a set of available sizes.

Critical design constraint. In the legacy system these eighteen names were written into the source code as literal text **364 times across four files**. The business has told us new products are coming. In your build, the catalogue is **data in a table, editable through the interface by the owner**. Adding a product must never require a code change. This is the single largest structural failure of the previous attempt and the clearest test of whether the rebuild succeeded.

1.4 How each item is priced — verified from source

These formulas are extracted from the legacy billing code and are correct as written. They must carry over exactly. **[Verified]**

SALE FORM	PRICING FORMULA	NOTES
Rolls	<code>rate × size × quantity</code>	Size (width) is a multiplier — rate is per inch of width, per roll. Easy to get wrong.
Reels	<code>rate × weight_kg</code>	
Packets	<code>(length × width × grammage ÷ 15500) × packets × rate</code>	Standard ream-weight conversion. The most valuable line in the codebase.
Totay	<code>rate × weight_kg</code>	
Jutta	<code>rate × weight_kg</code>	
Raddi	<code>rate × weight_kg</code>	

SALE FORM	PRICING FORMULA	NOTES
Nali	<code>rate × weight_kg</code>	

The constant **15500** converts square inches to the ream basis used in the trade. Do not “simplify” it.

1.5 Customers

Two kinds, and the difference drives the whole billing flow. **[Verified]**

	CASH CUSTOMER	LEDGER CLIENT
ID format	Prefixed <code>c</code> (e.g. <code>c1</code>)	Plain integer (e.g. <code>1</code>)
Pays	At the counter, at time of sale	On account, settled later
Identified by	Name plus contact number — two customers with the same name are kept separate by phone number	Registered client record
Needs	Fast billing, printed receipt	Running balance, statements, recovery follow-up
Cross-reference	—	Records the page number in the paper register, so digital and manual can be reconciled

1.6 The money model

Every ledger entry carries these fields. **[Verified]**

`DATE` · `RECEIPT_NUMBER` · `CLIENT_ID` · `CLIENT_NAME` · `CONTACT_NO` · `DETAILS_OF_BILL` · `DEBIT` · `CREDIT` · `CREDIT_DETAILS` · `RENT` · `BALANCE` · `PAGE_ON_MANUAL_LEDGER`

- **Debit** increases what the customer owes (a sale).
- **Credit** reduces it (a payment). `CREDIT_DETAILS` records the method — cash, cheque, transfer.
- **Rent** is a separate charge with unresolved meaning. **[Verify]**
- **Balance** is the running total.

[Decision] Never store the balance. The legacy system stored a computed `BALANCE` in every row. Once any earlier row is edited or deleted, every later balance is silently wrong. Calculate the balance on read, always. Store only debit, credit and rent.

2. The processes, as they run today

Twelve processes. For each: what happens now, what hurts, and what automation is required.

Everything in this section is manual today — paper registers, a physical ledger, WhatsApp and phone calls.

P1 — Order taking

Who: the owner, occasionally senior staff. **How now:** orders arrive by phone call, WhatsApp message or a customer walking in. The owner notes them down, or simply remembers them. **[Verify]** **Pain:** orders live in the owner's head and in scattered WhatsApp threads. Nothing tracks whether an order was actually fulfilled. An order missed is revenue lost and a customer annoyed. **Automation required:** capture orders into a structured list without forcing the owner to fill in a form. He should be able to dictate or paste a rough list in his own words and get back a clean, confirmable set of order lines. → **Agent A1**

P2 — Dispatch and gate pass

Who: loading staff and whoever controls the gate. **How now:** goods are loaded and leave. A gate pass may or may not be written. **[Verify — this is the biggest unknown in the whole document.]** **Pain:** this is the point where physical stock actually changes, and it is the least recorded. If goods leave without a matching bill, stock and money silently diverge. **Automation required:** a live dispatch board for the day showing every order line and its status, updated by a human as things physically happen. → **Agent A2**

P3 — Cash billing at the counter

How now: walk-in customer, items totalled, cash taken, receipt written. **Pain:** manual arithmetic on seven different pricing rules — the rolls formula alone ($\text{rate} \times \text{size} \times \text{qty}$) is a common error. Receipt numbers are hand-assigned and collide. **Automation required:** fast billing screen with the pricing rules built in; a receipt number that cannot duplicate; printed and WhatsApp receipt. → **Module, assisted by Agent A4**

P4 — Client billing on account

How now: goods supplied, entry written into that client's page in the manual ledger. **Pain:** balances are computed by hand down a column and drift. Finding a client's current position means flipping through a register. **Automation required:** same billing engine as P3, but posting a debit to the client's account instead of taking cash. → **Module, assisted by Agent A4**

P5 — Recording payments received

How now: a credit line written into the ledger, with the method noted. **Pain:** payments recorded late or not at all, so recovery chases customers who have already paid. **Automation required:** simple payment entry against a customer, with method and reference; balance updates immediately.

P6 — Stock in (purchase and delivery)

How now: paper arrives from a vendor, entered into the manual stock register with type, size, weight or quantity, vendor and rate. **[Verified — this is the shape of the legacy `reels_stock` and `rolls_stock` data.]** **Pain:** entered late, or in a different unit than it will be sold in. **Automation required:** stock-in entry, plus the ability to photograph a delivery challan and have it read. → **Agent A3**

P7 — Stock out (consumption)

How now: implicit — stock leaves when it is sold or converted, and the register is adjusted afterwards, if at all. **Pain:** the register lags reality, so nobody trusts the stock figure, so nobody maintains it. A self-reinforcing failure. **Automation required:** stock out must be a **consequence** of a recorded event (a gate pass or a bill), never a separate thing someone remembers to do. → **See Section 3**

P8 — Stock reconciliation

How now: occasional physical count against the register. **[Verify — frequency]** **Pain:** slow, and variances are absorbed rather than investigated. **Automation required:** photograph the physical count sheet, compare against system stock, produce a variance report, propose adjustments for approval. → **Agent A5**

P9 — Expenses

How now: daily expenses noted, often several on one line. The legacy system concatenated them into a single cell separated by `||`, with one total. **[Verified]** **Pain:** unanalysable — you cannot ask what fuel cost last month. **Automation required:** one row per expense with a category; still allow the owner to enter them as one free-text line and have it split automatically. → **Agent A6 assists**

P10 — Daily report / day close

How now: at day end, totals for cash in, credit given, expenses and net position are compiled by hand into a fixed format. **[Verified — the legacy `dailyreportv2.py` computes exactly this.]** **Pain:** takes time, done from memory and scraps, and errors compound because tomorrow starts from today's closing figure. **Automation required:** generated automatically from the day's entries, in the format the owner already recognises, delivered to his phone. Exceptions narrated, not just numbers listed. → **Agent A6**

P11 — Recovery / outstanding balances

How now: the owner works out who owes what by reading the ledger, then calls or messages them. **Pain:** entirely dependent on the owner's attention. Old debts go quiet. **Automation required:** ageing view of receivables, and drafted reminder messages in the customer's language for the owner to approve and send. → **Agent A7**

P12 — Production / conversion

How now: a reel or tota is cut into packets to a customer's specification. **Pain:** input consumed and output produced are not linked, so wastage is invisible. **[Verify — whether they want this tracked at all.]**

Automation required: possibly a conversion record — input consumed, output produced, wastage. **Treat as out of scope for the first release** unless the factory says otherwise.

3. The core rule: one transaction, many effects

This section is the heart of the system. If you get nothing else right, get this right.

The 2022 system failed partly because billing, stock and the ledger were three separate screens writing to three separate places, at different times, with no guarantee they agreed. **They are not three things. They are one event with three consequences.**

3.1 What an approved bill must do — atomically

When a bill is approved, all of the following happen together, or none of them happen:

1. **Ledger** — a debit is posted to the customer's account (or cash is recorded as received).
2. **Stock** — quantities are decremented for every line, and a stock movement row is written for each, referencing this bill.

3. **Order** — the originating order lines are marked fulfilled.
4. **Documents** — the receipt is generated, numbered from a sequence that cannot produce duplicates.
5. **Audit** — who approved it, when, and whether an agent proposed it.

If step 2 fails because stock is insufficient, step 1 must not have happened. In the legacy system these were separate spreadsheet writes and routinely diverged. Use a database transaction. This is not optional.

3.2 The document flow

There are two paths, and the difference matters.

Path A — Counter sale (cash)

```
Order (optional) → Bill → Payment → Goods leave with receipt
```

Everything happens in one moment. The receipt is the gate pass.

Path B — Delivery to a ledger client

```
Order → Confirm → Load → GATE PASS (goods leave) → Bill → Ledger debit
```

Goods physically leave **before** the bill exists. This is the gap where stock and money diverge, and it is the gap the dispatch tracker is there to close.

3.3 [Decision] Stock decrements at the gate pass, not at the bill

The legacy system decremented stock when the bill was raised. On Path B that is wrong: the paper left the building hours or days earlier.

Recommendation: stock moves when goods physically move — at the gate pass. The bill then references the gate pass, and on approval reconciles against it. If the billed quantity differs from what left the gate, the system raises a variance for a human to resolve rather than silently overwriting.

Consequence: you need a `gate_passes` table and a gate-pass state on every order line. This is more work than the legacy design and it is worth it — it is the only way the stock figure ever becomes trustworthy.

3.4 Document state machine

Every order line moves through these states. The dispatch board in Agent A2 is a view of this.

STATE	MEANING	WHO MOVES IT FORWARD
draft	Parsed from the owner's input, not yet confirmed	Agent A1 proposes
confirmed	Owner has approved it as a real order	Owner
allocated	Stock reserved against it	System
loaded	Physically loaded, ready to leave	Staff

STATE	MEANING	WHO MOVES IT FORWARD
gate_passed	Left the premises; stock decremented here	Gate staff
billed	Bill raised and approved	Counter / owner
settled	Paid in full	System, on payment
cancelled	Called off; any reservation released	Owner
variance	Billed and dispatched quantities disagree	Requires human resolution

Backward transitions must be possible and must be logged. Factories change their minds; the system must not fight them.

4. Target system architecture

4.1 Modules

MODULE	COVERS	PRIORITY
Catalogue	Paper types, sizes, rates, units, descriptions	1
Customers	Cash and ledger customers, contacts, credit terms	1
Billing	Seven sale forms, pricing engine, receipts	1
Ledger	Debits, credits, rent, computed balances, statements	1
Stock	Levels, stock-in, movements, valuation	2
Orders & Dispatch	Order lines, gate passes, the dispatch board	2
Expenses	Categorised expense entry	3
Reports	Daily report, receivables ageing, stock valuation	3
Admin	Users, roles, audit trail, backups	3

4.2 Data model

Twenty-nine spreadsheet sheets collapse into roughly ten tables. Note that **adding a customer or a product no longer changes the structure of anything** — that alone removes most of the legacy system's failure modes.

TABLE	REPLACES	KEY POINTS
customers	Sheets 1 ... 4 and the cash roster	Type column distinguishes cash from ledger
ledger_entries	cash table and every per-customer sheet	Debit, credit, rent. No stored balance
item_types	18 names hardcoded in 4 files	Editable by the owner. Unit of measure per type
item_variants	The per-type sheets	Type × size, with rate
stock_items	reels_stock, rolls_stock, totay	Current position by type, size, unit
stock_movements	The three *_stock_in_out sheets	Every in and out, linked to its cause
orders / order_lines	Nothing — did not exist	The dispatch board reads from here
gate_passes	Nothing — did not exist	Links physical dispatch to the bill
bills / bill_lines	Bill details buried in ledger text	Line-level detail, so reporting works
expenses	BookExpense.xlsx	One row per expense, categorised
users	Nothing — did not exist	Roles and permissions
audit_log	Nothing — did not exist	Human and agent actions both recorded
agent_runs	Nothing	Every agent invocation, input, output, decision

4.3 Non-negotiable platform requirements

- **Bilingual English / Urdu from line one.** Every label is a translation key from the first commit. Layout handles right-to-left. Printed receipts carry an Urdu font. Retrofitting this later costs three times as much.
- **Production hosting with automated daily backups and point-in-time recovery.** The absence of backups is what turned the 2022 data corruption into a total loss.
- **Works on a phone.** The owner checks figures on his phone; staff may bill on one.
- **Tolerant of bad connectivity.** At minimum, a billing screen that does not lose entered data when the connection drops. **[Verify — how good is the factory's internet?]**
- **Every screen has a manual override.** See Section 0.

5. The agents

This is the work you are principally being hired for.

5.1 The governing rule

Agents propose. Humans dispose.

No agent writes to the ledger, moves stock, or sends a message to a customer on its own authority. An agent produces a **draft** — a proposed set of entries, clearly presented, that a human accepts, edits or rejects with one action. Read-only agents (answering questions, drafting reports for review) may run freely.

This is not timidity. It is what makes the system adoptable: the owner stays in control of his own business, and every automation failure degrades into a small annoyance instead of a wrong balance on a customer's account.

5.2 Agent roster

ID	AGENT	TYPE	HUMAN CHECKPOINT
A1	Order Intake	Write (proposes)	Owner confirms order lines
A2	Dispatch Tracker	Coordination	Staff update status; agent only nudges
A3	Document Capture	Write (proposes)	Side-by-side verification before posting
A4	Billing Assistant	Write (proposes)	Approve to post
A5	Stock Reconciliation	Write (proposes)	Owner approves each adjustment
A6	Day Close	Read + draft	Owner reviews before it is sent
A7	Receivables	Read + draft	Owner approves every message before sending
A8	Catalogue & Rates	Write (proposes)	Owner approves rate changes
A9	Business Query	Read only	None required

A1 — Order Intake Agent

The requirement in the owner's words: *"these are the orders for today that need to be sent."*

FIELD	SPECIFICATION
Trigger	Owner sends a voice note, a typed list, a WhatsApp forward, or a photo of a handwritten list
Input	Unstructured text or audio, in English, Urdu or Roman Urdu, using trade shorthand

FIELD	SPECIFICATION
Tools	Customer lookup (fuzzy), catalogue lookup (fuzzy), stock availability check, order draft creation
Output	A set of draft order lines: customer, item type, size, unit, quantity, and a confidence score for each
Human checkpoint	Owner reviews the parsed list side by side with his original input and confirms, edits, or rejects
Guardrails	Never invents a customer or a product. Unmatched names go to an “unresolved” bucket for the owner to map — and the mapping is remembered
Failure mode	Ambiguous input becomes a flagged line, never a guess. Blocking nothing

Why this is harder than it looks. The owner will write things like *“Rathore ko 17 fluting 10 roll, aur Bilal 20 kg raddi”*. You are parsing mixed-script, mixed-language trade shorthand with abbreviated customer names. Build an alias table from day one: every time the owner corrects a match, store it. Within weeks the agent will resolve most names on the first try.

Success criterion: the owner can dictate a day’s orders in under a minute and confirm the parsed result with at most one or two corrections.

A2 — Dispatch & Gate Pass Tracker Agent

The requirement: *“the tracker should be there to see — there should be a human in the loop that updates those entries, whether it is sent or not, whether via billing or through a gate pass.”*

FIELD	SPECIFICATION
Trigger	Continuous through the working day
Input	Confirmed order lines and their current states
Tools	State transitions, stock reservation, gate pass creation, notification
Output	A live dispatch board for the day; end-of-day summary of what did not go out and why
Human checkpoint	Humans move every state forward. The agent never marks anything dispatched by itself
Guardrails	Cannot mark <code>gate_passed</code> or <code>billed</code> . Cannot cancel an order
Failure mode	If nothing is updated all day, it escalates to the owner — it does not assume

What the agent actually contributes. The board itself is a screen, not an agent. The agent’s value is in watching it:

- Order confirmed three hours ago and still not loaded → nudge the staff.
- Goods gate-passed yesterday, still not billed → flag to the owner. *This is the leak that loses the most money.*
- Order line needs stock that is not there → flag before loading, not at the gate.
- Billed quantity differs from gate-passed quantity → raise a **variance** for someone to resolve.
- End of day: “Four lines still open. Two waiting on stock, one has been out on gate pass since Tuesday unbilled.”

Design note. Staff must be able to update status in one tap from a phone, in Urdu, standing at the gate with dirty hands. If it takes a form, it will not be used, and the whole tracker is worthless.

Success criterion: at any moment the owner can see, on his phone, exactly what has gone out today and what has not.

A3 — Document Capture Agent (vision)

The requirement: “a new guy basically takes the manual ledger, the manual stock, the daily report format, makes the necessary changes by maybe taking pictures and sending it to an AI agent.”

This is the bridge between the paper factory and the digital one, and it is what makes parallel running (Section 7) practical.

FIELD	SPECIFICATION
Trigger	Someone photographs a page and sends it, from the app or over WhatsApp
Input	A photo of a manual ledger page, a stock register page, a delivery challan, a daily report sheet, or a handwritten order list
Tools	Vision model, customer and catalogue lookup, draft entry creation
Output	Structured rows with a per-field confidence score, plus the source image retained alongside
Human checkpoint	Mandatory. A verification screen showing the photograph beside the extracted rows. Low-confidence fields highlighted. Nothing posts until a human confirms
Guardrails	Never posts automatically, regardless of confidence. Always keeps the source image linked to the entry
Failure mode	Unreadable page → asks for a better photo, naming what it could not read

Handle these four document types:

1. **Manual ledger page** — date, particulars, debit, credit, balance, in Urdu or English handwriting. The hardest. Note that the legacy ledger records a `PAGE_ON_MANUAL_LEDGER` field precisely so digital and paper can be tied together — use it.
2. **Stock register page** — type, size, quantity or weight.
3. **Delivery challan from a vendor** — a stock-in event.
4. **The daily report sheet** — used to seed the report format the owner already recognises.

Build a correction loop. Every human correction on the verification screen is training data. Log the image region, what the model read, and what the human changed it to. After a few hundred corrections you will know exactly where the model is weak, and you will have an evaluation set that reflects this factory’s actual handwriting rather than a generic benchmark.

Success criterion: a page of the manual ledger can be digitised and verified faster than it could be typed in.

A4 — Billing Assistant Agent

FIELD	SPECIFICATION
Trigger	A gate pass is ready to bill, or an order line is complete, or a counter sale is being entered
Input	Gate pass or order lines, customer record, current rates
Tools	Pricing engine, stock check, customer credit position, bill draft, receipt generation
Output	A complete draft bill with every line priced
Human checkpoint	Approval posts the bill. This is the atomic transaction of Section 3.1
Guardrails	Cannot approve its own bill. Cannot alter a rate — it flags and asks. Must warn before exceeding a customer’s credit limit
Failure mode	Missing rate or ambiguous item → drafts what it can and flags the rest

What it should catch before a human sees it:

- Billed quantity does not match the gate pass.
- The rate used is not the catalogue rate for this customer.
- Stock would go negative.
- The customer is already over their credit limit, or has an overdue balance.
- The rolls formula is being applied without a size — a genuine source of error, since `rate × size × qty` is not obvious.

Note on receipt numbers. The legacy system generated the next number by reading the last row of a sheet and adding one, drawing cash and client bills from the same counter. This produces duplicate numbers on real customer paperwork. Use a database sequence. **[Verified — this bug exists in the current code.]**

A5 — Stock Reconciliation Agent

FIELD	SPECIFICATION
Trigger	A physical count is performed, or on a schedule
Input	Photographed count sheets (via A3) or direct entry, plus current system stock
Tools	Stock query, movement history, adjustment proposal
Output	A variance report — expected vs counted, by type and size, with a likely explanation for each gap
Human checkpoint	Owner approves each adjustment individually. Large variances require a reason to be recorded
Guardrails	Cannot adjust stock unilaterally. Cannot adjust historical movements — only post a dated correction
Failure mode	Cannot explain a variance → says so plainly rather than inventing a cause

The valuable behaviour is not computing the difference — it is explaining it. *“Fluting 17 is short by 40 rolls. Three gate passes issued last week were never billed; two of them are for this type.”* That turns a number into an action.

A6 — Day Close Agent

FIELD	SPECIFICATION
Trigger	End of day, on a schedule, or on request
Input	The day’s bills, payments, expenses, stock movements and open orders
Tools	Read across all modules, report generation, WhatsApp draft
Output	The daily report in the format the factory already uses , plus a short narrative of what needs attention
Human checkpoint	Owner reviews before it is circulated
Guardrails	Read-only over transactional data. Does not close or lock anything

FIELD	SPECIFICATION
Failure mode	Incomplete data → reports it as incomplete and names what is missing

Match the existing report format first. Get the paper version photographed (Section 9) and reproduce it. An owner who has read the same layout for years will not adopt a better one; he will simply stop reading it.

Also assign this agent the expense-splitting job from P9: the owner types “*diesel 3000, chai 400, loading 1200*” and it proposes three categorised rows.

A7 — Receivables Agent

FIELD	SPECIFICATION
Trigger	Weekly, or on request
Input	All ledger balances with ageing
Tools	Ledger query, message drafting, WhatsApp link generation
Output	Ageing list, and a drafted reminder per customer in their language
Human checkpoint	The owner approves every single message before it sends. No exceptions
Guardrails	Never messages a customer autonomously. Never states a balance it has not verified against the ledger
Failure mode	Disputed or unclear balance → excluded from the run and flagged

Tone matters commercially. A reminder to a twenty-year customer is not the same message as one to a defaulter. Let the owner set a tone per customer — polite, firm, final — and draft accordingly.

A8 — Catalogue & Rate Agent

FIELD	SPECIFICATION
Trigger	Owner adds a product or changes rates, possibly from a photo of a supplier rate list
Input	Free-text or photographed rate information
Tools	Catalogue read and write proposal, historical rate lookup
Output	Proposed catalogue additions or rate changes, showing old vs new

FIELD	SPECIFICATION
Human checkpoint	Owner approves. Rate changes are dated and never overwrite history
Guardrails	Cannot delete a product that has transaction history — deactivates it instead

This is the agent that delivers on “*there will be new products as well.*” Adding a product must be a conversation, not a code change.

A9 — Business Query Agent

FIELD	SPECIFICATION
Trigger	Owner asks a question, in any language, from his phone
Input	Natural language — “ <i>Rathore kitna deta hai?</i> ”, “ <i>how much Fluting 17 is left?</i> ”
Tools	Read-only access across every module
Output	A direct answer with the figures it used
Human checkpoint	None — read-only
Guardrails	Read-only, enforced at the database connection, not by prompt. Always shows its working. Says “I don’t know” rather than estimating

The cheapest agent to build and probably the one the owner will use most. Do not leave it to the end.

6. Engineering rules for the agent layer

These are not suggestions.

1. Agents call the same services the UI calls. No agent gets direct database access. Pricing, stock movement and ledger posting live in one domain layer that both the screens and the agents use. Two implementations of the rolls pricing formula means two different answers, and the customer will find the discrepancy before you do.

2. Every agent write is a proposal object. Persist proposals as data with a status of `pending`, `accepted`, `rejected` or `edited`. Never let an agent call a mutation directly. This gives you the audit trail, the undo, and the training signal in one structure.

3. Log every agent run. Input, model, prompt version, output, tokens, latency, what the human did with it. Without this you cannot tell whether a prompt change made things better or worse.

- 4. Build the evaluation set before you build the agents.** Twenty real order dictations, thirty real ledger photographs, fifteen real rate lists. Every prompt change runs against them. This is the difference between engineering and guessing, and on document capture it is the only thing that will keep you honest.
- 5. Money and stock require a human. Always.** No configuration flag may bypass this, however confident the model is or however much the owner asks for it later.
- 6. Read-only means read-only at the connection level.** A9’s database credentials must not permit writes. Prompt instructions are not a security boundary.
- 7. Fail into a queue, never into an error.** An agent that cannot parse something creates a “needs attention” item. It never blocks a sale, and it never discards the input.
- 8. Costs are per-run and visible.** Voice notes and document photographs are the expensive paths. Surface a monthly figure so the owner can see what he is paying for.
- 9. The catalogue is data.** If any agent prompt contains a hardcoded list of the eighteen paper types, the rebuild has reproduced the original system’s central defect.

7. Adoption: running in parallel

The factory will not switch overnight, and should not be asked to.

STAGE	WHAT HAPPENS	EXIT CONDITION
1. Shadow	Manual register stays primary. Staff also enter into the system, or photograph pages via A3	System matches the register for five consecutive working days
2. Dual	System is primary for billing; register kept as backup and reconciled daily	Zero unexplained variances for two weeks
3. Primary	System is the record. Register kept for customer-facing continuity only	Owner is comfortable
4. Sole	Register retired at the owner’s discretion	—

Do not skip Stage 1. It is where you discover what the process actually is, as opposed to what this document says it is. Expect to find that at least two of the twelve processes work differently from what is written here.

Never delete the manual ledger data. Photograph it, digitise it, and keep the images.

8. Delivery plan

Each phase is independently deployable and useful on its own.

Phase 1 — Foundation. Repository and version control from the first commit. Database schema. Authentication and roles. Bilingual framework. Deployment pipeline with automated backups. Catalogue seeded with the eighteen types, their sizes and current rates. Domain services for pricing, stock and ledger — the layer agents will later call. *Acceptance:* a new product can be added through the interface with no code change.

Phase 2 — Billing and ledger. Customers. All seven sale forms with verified pricing. Receipt numbering from a sequence. Bill posting as one atomic transaction. Printed receipt and WhatsApp send. Opening balances entered from the register. Agent A9 (query) — cheap, and it starts building the owner's trust early. *Acceptance:* a full day's billing runs on the system, and the totals match the manual register exactly.

Phase 3 — Stock, orders and dispatch. Stock levels and movements. Stock-in. Orders, gate passes and the dispatch board. Automatic stock movement on gate pass. Agents A1 (order intake), A2 (dispatch), A4 (billing assistant). *Acceptance:* the owner dictates a day's orders and every line is tracked from order to bill without a spreadsheet.

Phase 4 — Capture and reports. Agent A3 (document capture), with the verification screen and the correction loop. Expenses. Daily report in the factory's own format. Receivables ageing. Agents A5, A6, A7. *Acceptance:* a manual ledger page is digitised and verified faster than it could be typed.

Phase 5 — Hardening and handover. Verified backup **and restore** — test the restore, not just the backup. Audit trail. Roles and permissions. Agent evaluation suites. A week on site with the staff, fixing what real use exposes. Documentation and training in Urdu. *Acceptance:* the factory runs a full week on the system with no reconciliation variances and no fallback to paper for billing.

Phase 5 is the phase the 2022 attempt skipped. It is the reason there is no working system today. Budget for it properly.

9. What to collect from the factory before writing code

You cannot design these screens from this document alone. Get these first — a day spent collecting them will save weeks.

1. **Photographs of the manual stock register** — several pages. Shows the columns actually kept and the shorthand staff write.
2. **Photographs of the manual customer ledger** — several pages, including a busy account. This is A3's primary training material.
3. **A photograph of the daily report format** currently in use. A6 must reproduce it.
4. **A photograph of a real customer receipt**, and a gate pass if one exists. The printed output is what customers see; it should match what they already recognise.
5. **The new products** — names, sizes, units, and whether each prices by weight, count or area.
6. **Current rates** for every type and size. Those in `book.xlsx` are four years stale.
7. **What rent means.** It has its own column in every ledger sheet, so it matters, but nothing in 12,000 lines of code explains it. Ask directly.
8. **How gate passes work today** — is one written? Who signs it? Does it precede the bill? Section 3.3 depends entirely on the answer.

9. **Who uses the system, on what devices, and how reliable the factory internet is.** Determines how much offline tolerance the billing screen needs.
10. **Credit terms** — do ledger clients have limits? Agreed payment periods?
11. **A recording or transcript of the owner dictating a typical day's orders.** This is A1's evaluation set, and it is the single most useful artefact on this list.

10. Definition of done

The build is complete when all of the following are true:

- A new paper type can be added by the owner through the interface, in under a minute, with no code change.
- A bill cannot be posted without stock and ledger updating in the same transaction.
- Two bills raised at the same moment cannot receive the same receipt number.
- The owner can dictate a day's orders and confirm the parsed result in under two minutes.
- Every order line's journey from order to gate pass to bill is visible on one screen.
- A page of the manual ledger can be photographed, verified and posted faster than it could be typed.
- No agent can move money or stock without a human approving it.
- A backup has been restored to a working system in a test, and the test is documented.
- The daily report matches the format the owner already reads.
- The interface works in both English and Urdu, including printed receipts.
- The factory has run a full week on the system without falling back to paper for billing.

Appendix A — Verified pricing formulas

Extracted directly from `CashBillclass.py` in the legacy codebase. Reproduce exactly.

ITEM	FORMULA	SOURCE
Rolls	<code>rate × size × quantity</code>	<code>add_rolls()</code>
Reels	<code>rate × weight_kg</code>	<code>add_reels()</code>
Packets	<code>(length × width × grammage ÷ 15500) × packets × rate</code>	<code>add_packets()</code>
Totay	<code>rate × weight_kg</code>	<code>add_totay()</code>
Jutta	<code>rate × weight_kg</code>	<code>add_jutta()</code>
Raddi	<code>rate × weight_kg</code>	<code>add_raddi()</code>
Nali	<code>rate × weight_kg</code>	<code>Add_Nali()</code>

Appendix B — Legacy codebase map

Two folders exist. **Factory Management System First Deployment** is the newer and more complete version, despite the other being named “Final”. It was last edited 24 September 2022, six days after “Final”, and it alone contains the Google Drive backup scripts and the stock record generator. Use it as the reference.

FILE	LINES	WHAT TO READ IT FOR
<code>CashBillclass.py</code>	2,966	The pricing formulas. The most valuable file
<code>factorycustomers.py</code>	3,831	Customer and ledger structure, statement generation
<code>stock.py</code>	1,917	Stock model. Also the clearest example of what not to do
<code>dailyreportv2.py</code>	865	What belongs on the daily report
<code>stock_in_out_record.py</code>	454	Movement history structure
<code>expense.py</code>	400	Expense structure
<code>book.xlsx</code>	29 sheets	The real data shapes. Read this before the code
<code>readme.txt</code>	—	The original author’s own notes, including the ID rules

Read these files for domain knowledge only. Do not port the code. It is PyQt5 desktop code with business rules embedded in button handlers, coordinated by 376 `global` statements, writing to Excel 127 times.

Known defects — do not reproduce

DEFECT	CONSEQUENCE
Excel used as a database, one sheet per customer	Data corruption. Documented in the factory’s own <code>readme.txt</code>
364 hardcoded paper-type strings across 4 files	Adding a product takes days
Stored <code>BALANCE</code> column	Silently wrong after any edit
Receipt number = last row + 1	Duplicate receipt numbers on customer paperwork
Stock decremented at billing, not at dispatch	Stock never matches reality
<code>pywhatkit</code> for WhatsApp	Simulates keystrokes in a browser. Breaks constantly
Fixed 1360×850 window, bundled 2018 Windows binaries	Cannot be installed or used on a phone

DEFECT	CONSEQUENCE
No version control, no tests, no backups	No recovery when it broke

Appendix C — Open questions

Every **[Verify]** marker in this document, collected. Resolve these in the first week.

1. What does `rent` charge for?
2. Is a gate pass written today? Who issues and signs it? Does it precede the bill?
3. What exactly are `jutta` and `nali`?
4. How often is a physical stock count done?
5. Should production and conversion (reel → packets) be tracked, including wastage?
6. Do ledger clients have credit limits and agreed payment terms?
7. How are orders received today — phone, WhatsApp, in person, and in what proportion?
8. How reliable is the factory's internet connection?
9. Who will use the system, and what are their roles and permissions?
10. Are there vendors and purchase records to track, or is stock-in entered without a supplier ledger?

Prepared from a direct reading of both legacy codebases and the `book.xlsx` and `BookExpense.xlsx` workbooks. All figures cited as [Verified] are counted from source, not estimated.